GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

Nexus：面向机器人与训练数据生成的新型跨平台 GPU Rust多物理场仿真器

Nexus: A New Cross-Platform GPU-Based Rust Multiphysics Simulator for Robotics and Training Dataset Generation



Crozet Sébastien

Dimforge创始人

Founder, Dimforge



GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

Nexus: A New Cross-Platform GPU-Based Rust Multiphysics Simulator for Robotics and Training Dataset Generation

Sébastien Crozet

sebcrozet@dimforge.com



Open-source Rust developer since 2013, founder of Dimforge

Sébastien Crozet

Founder of Dimforge, France
github.com/dimforge

13 years of open-source Rust, since before Rust 1.0

94M crates.io downloads

19k+ GitHub stars across Dimforge projects

2013 — Rust before 1.0

- **nalgebra** — linear algebra
- **ncollide** — collisions
- **nphysics** — physics
- **kiss3d** — graphics

2019–20 — Dimforge creation

- **Rapier** — physics
- **Parry** — geometry
- **Salva** — fluids
- **rapier.js** — wasm

2022–25 — GPU experiments with Cuda, WGSL, Slang

- **sparkl** — CUDA MPM
- **wgsparkl** — WebGPU MPM
- **slosh** — Slang MPM

Today — A multi-platform, pure Rust, GPU stack

- **khal** — GPU abstraction
- **vortx** — GPU tensors
- **Nexus** — multiphysics

CONTENTS

Part 01 Ecosystem Review

Part 02 Introducing Nexus

Part 03 Performance & Benchmarks

Part 04 Nexus use-cases



01

Ecosystem Review

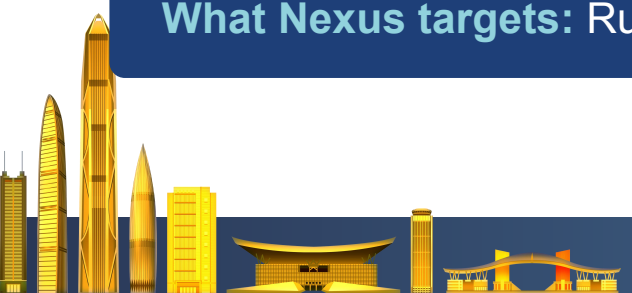


GPU simulators for robotics today

Powerful tools, but almost all are Python over a kernel DSL, and most are tied to NVIDIA

Simulator	Language / kernels	GPU support	Browser	Physics
Isaac Lab	Python over PhysX 5 (C++)	NVIDIA RTX only	No	Rigid, deformables
MuJoCo Warp / MJX	Python on Warp / JAX	Warp: NVIDIA only MJX: any XLA device	CPU only	Rigid (+ flex)
Newton	Python on NVIDIA Warp	NVIDIA only (CPU on macOS)	No	Rigid, cloth, MPM, VBD
Genesis World	Python on Quadrants (Taichi fork)	CUDA, ROCm, Metal, Vulkan	No	Rigid, MPM, SPH, FEM, PBD
Nexus	Rust, shaders in Rust (rust-gpu)	WebGPU (Vulkan, Metal, DX12), Metal, CUDA, CPU	**Yes**	Rigid, MPM (FEM next)

What Nexus targets: Rust language from host to shader, write once and run everywhere including the web.



Why not stick to Genesis or Newton?

Different technologies (Rust, open standards), more platforms, not limited to small scenes

Rust from host to shader

- Physics, GPU kernels and bindings in one language and type system: strong typing across the native <> gpu interface.
- Compiler errors instead of runtime errors (Python).
- Rust is arguably the future of AI coding.

Every GPU, including the browser

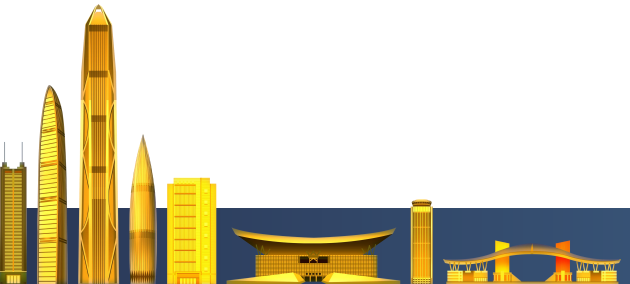
- WebGPU (Vulkan, Metal, DX12, browsers), native Metal, CUDA, and a CPU fallback.
- Newton is NVIDIA-only (CPU on macOS); Genesis needs Python + Quadrants.

Huge scenes or many environments

- 110k-body at ~15 ms per step on an M4 Max.
- Thousands of envs run in one dispatch.

Lightweight and embeddable

- No heavy Python dependencies / runtime or CUDA toolkit needed.
- But Python bindings remain available if needed.



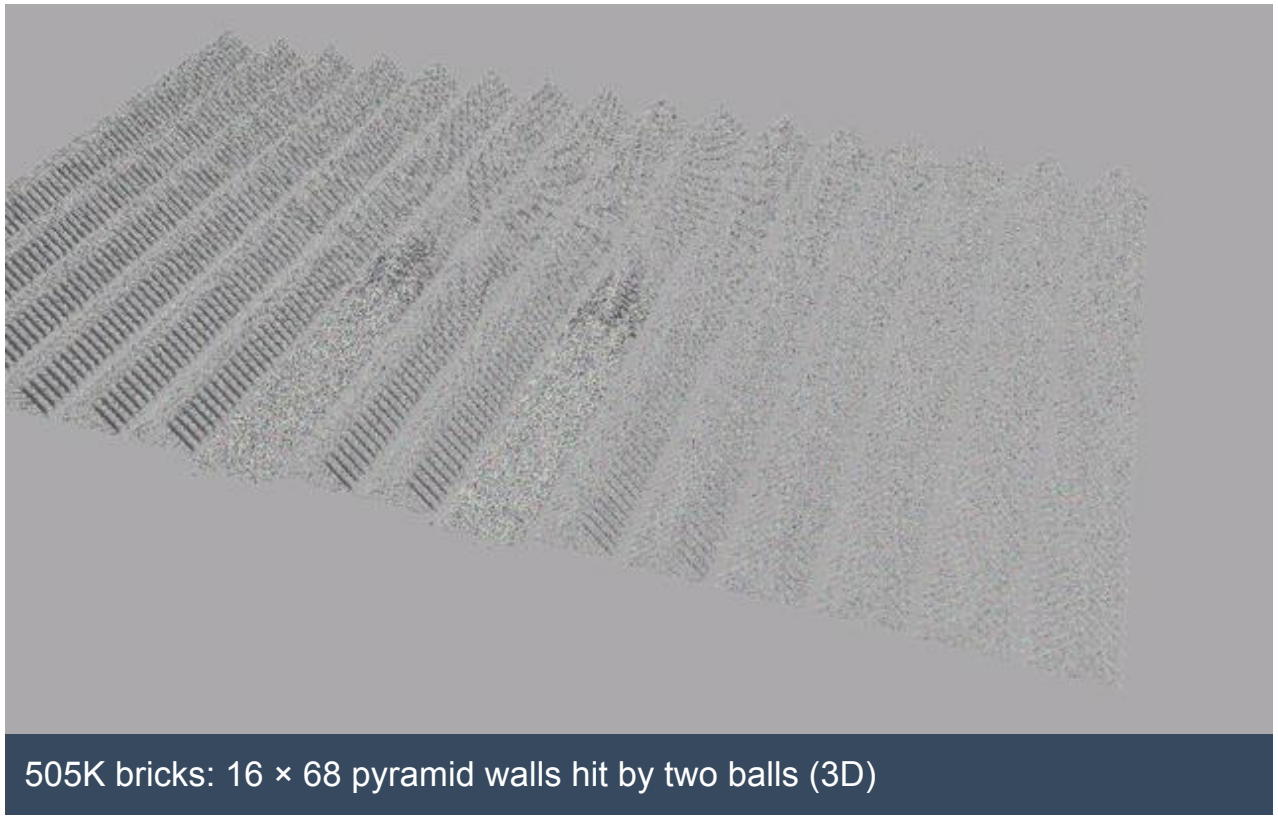
02

Introducing Nexus

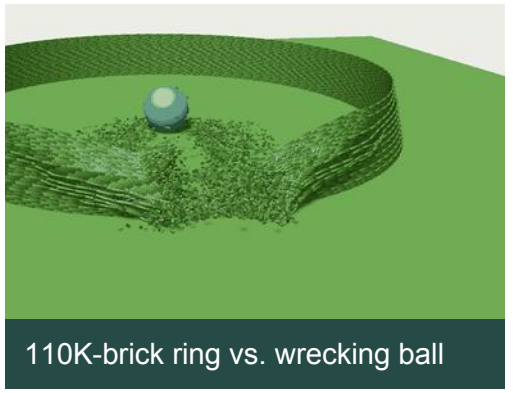


Nexus in action: rigid bodies

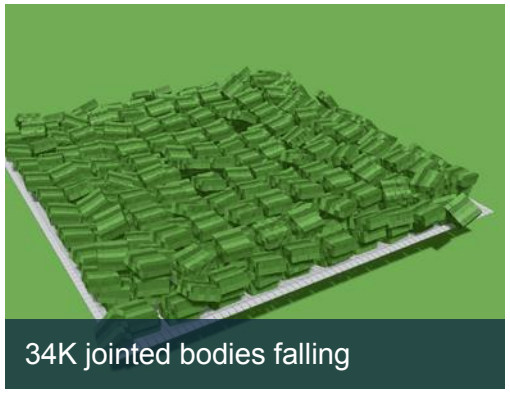
From 34K to 505K bodies in a single world, in 3D and in 2D



505K bricks: 16 × 68 pyramid walls hit by two balls (3D)



110K-brick ring vs. wrecking ball



34K jointed bodies falling



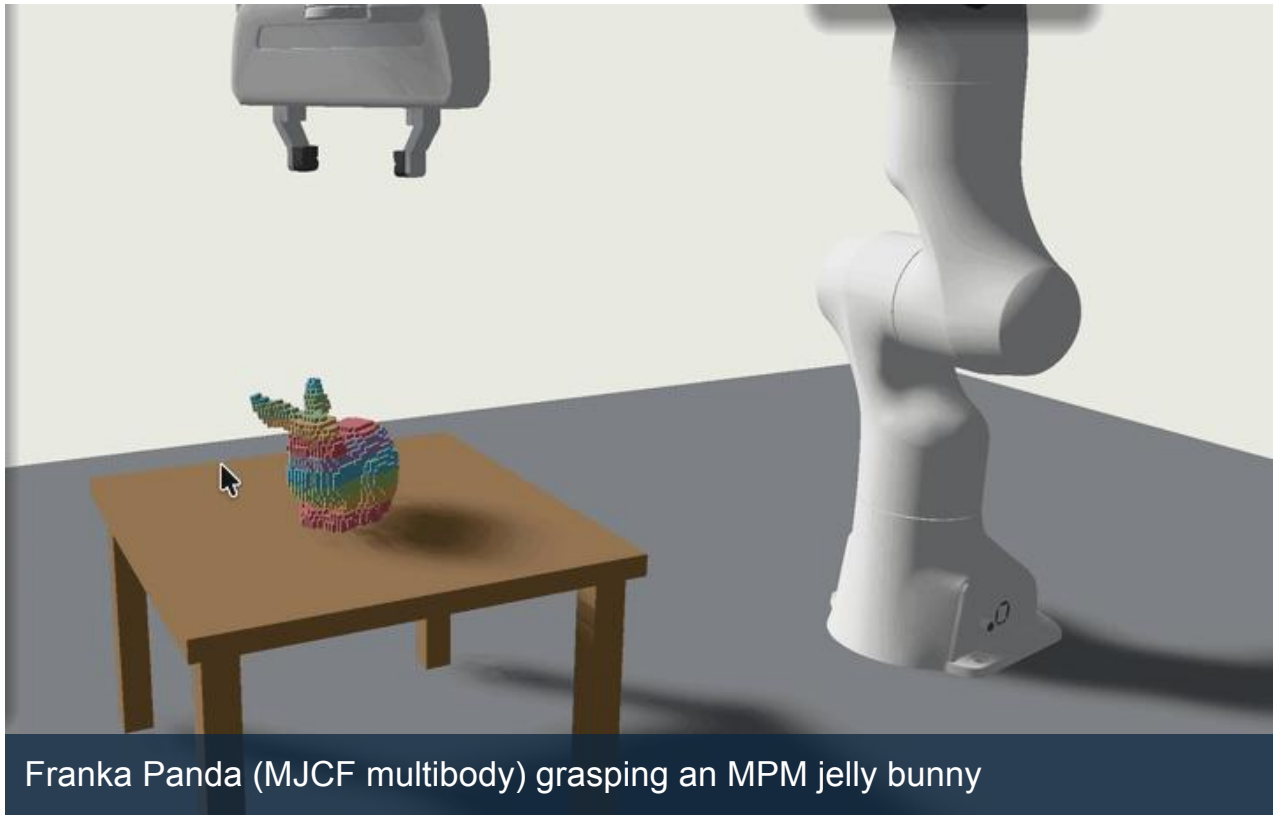
2D: wrecking ball vs. 40K boxes



2D: 100K-body jointed lattice

Nexus in action: robotics & MPM

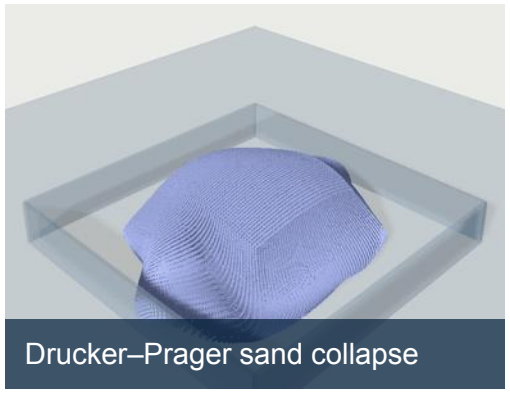
Articulated robots loaded from MJCF, and MPM materials interacting with rigid bodies



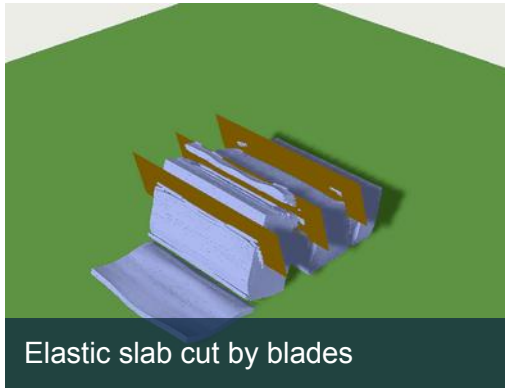
Franka Panda (MJCF multibody) grasping an MPM jelly bunny



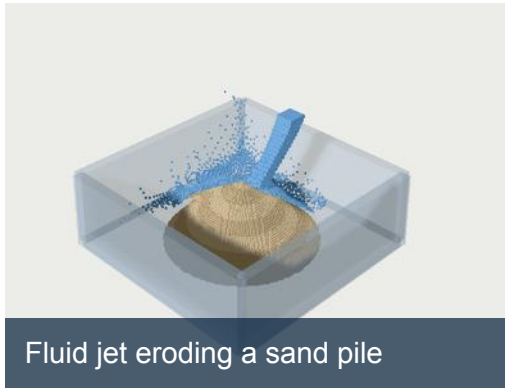
MuJoCo Menagerie robots (MJCF)



Drucker–Prager sand collapse



Elastic slab cut by blades



Fluid jet eroding a sand pile

Nexus architecture (high-level)

A layered stack: one Rust code base, compiled for every GPU backend

Applications
- RL training (zealot)
- Agentic robotics (AISLE)
- Showcases, replays
- Games, VFX

Interfaces
- Rust API: nexus2d / nexus3d
- Python: dimforge-nexus3d
- Browser: wasm + WebGPU
- Viewer: kiss3d

Nexus physics
- Rigid bodies, contacts, joints
- Multibodies (articulations)
- MPM: elastic, sand, snow, fluid
- Assets: URDF, MJCF, meshes

GPU libraries
- vortx: GPU tensors & linear algebra
- parry / rapier types shared with shaders

Portability layer

khal: buffers, dispatch, type-checked shader bindings

builds each kernel with
- rust-gpu → SPIR-V
- cuda-oxide / Rust-CUDA → PTX
- rustc → native CPU code

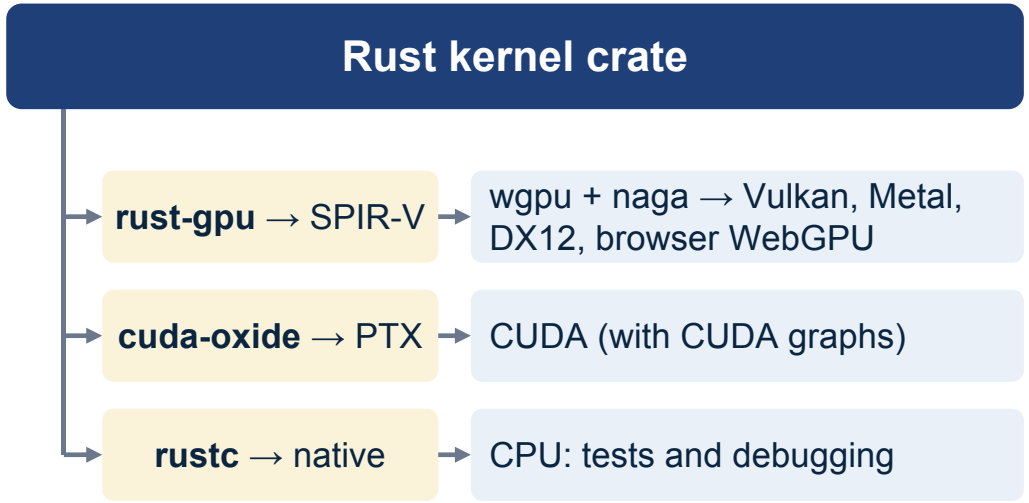
Backends
- WebGPU (wgpu): Vulkan, DX12, Metal, browsers
- Metal (native)
- CUDA
- CPU (debugging)

One Rust source, every GPU

Kernels are plain Rust functions; khal generates boilerplate for host-side dispatch.

```rust
// Shader crate (no_std), compiled by rust-gpu
#[spirv_bindgen]
#[spirv(compute(threads(64)))]
pub fn gpu_apply_solver_vels_inc(
    #[spirv(global_invocation_id)] id: UVec3,
    #[spirv(storage_buffer, ..., binding = 0)]
    solver_vels: &mut [Velocity],
    #[spirv(storage_buffer, ..., binding = 1)]
    solver_vels_inc: &[Velocity],
    #[spirv(uniform, ..., binding = 2)] batch_ids: &BatchIndices,
) {
    let i = id.x as usize;
    if i < batch_ids.num_bodies() {
        solver_vels.at_mut(i).linear += solver_vels_inc.at(i).linear;
    }
}

// Host crate: generated, type-checked wrapper
self.apply_solver_vels_inc.call(&mut pass, num_bodies,
    args.solver_vels, args.solver_vels_inc, args.batch_indices)?;
```



- **No layout drift:** host and GPU share the same structs.
- **Shaders are crates:** cargo, crates.io, docs, unit tests on CPU.
- **High-level language:** most Rust language constructs are available in the shaders.

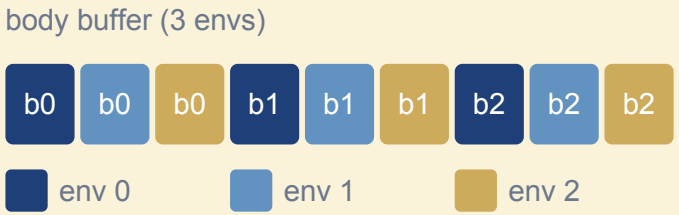
Nexus architecture (low-level): rigid bodies

Every stage of a rigid-body step runs on the GPU, for all environments at once

Broad phase	Narrow phase	Constraint prep	TGS-soft solver	Multibodies
• LBVH: Morton codes, GPU radix sort • Brute force for tiny scenes	• SAT + clipping for boxes • GJK / EPA + polygonal features • Trimeshes split per triangle	• Warm-start matching with last frame • Graph coloring (Topo-GC), seeded by last colors	• Substeps of colored PGS sweeps + stabilization • Joints: limits, motors	• Reduced coordinates • CRBA mass matrix + LU • Contacts, limits, motors in one PGS kernel

Multi-environment batching

- Buffers interleaved: global = local × N_envs + env
- One dispatch per kernel covers every env
- **Same topology** in every env



Opt-in determinism

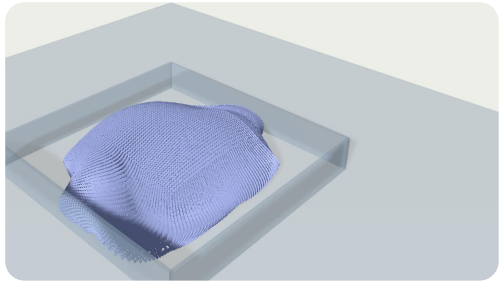
- Bit-exact replays on the same machine
- No float atomics; canonical contact order
- Overhead: +10–17% per step on heavy scenes

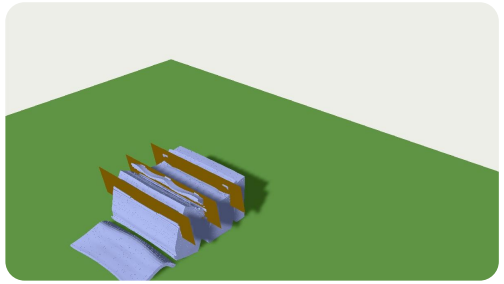
Nexus architecture (low-level): MPM

A sparse-grid MPM-MLS pipeline with rigid-body solver coupling

1	**Grid sort**	Sparse 4×4×4 blocks in a GPU hash map; counting sort
2	**P2G**	Shared-memory tiles, one thread per node: no float atomics
3	**Grid update**	Gravity, CFL velocity clamp, rigid collisions
4	**G2P**	APIC transfer, quadratic B-splines
5	**Particle update**	Constitutive model, chunked add/remove

Repeated every substep, for every particle





Materials

- Corotated elastic
- Neo-Hookean
- Drucker–Prager sand
- Snow
- Weakly compressible fluid

Coupling with rigid bodies

- **Today:** bodies push particles (one-way)
- **Next:** two-way impulses applied inside the TGS substeps

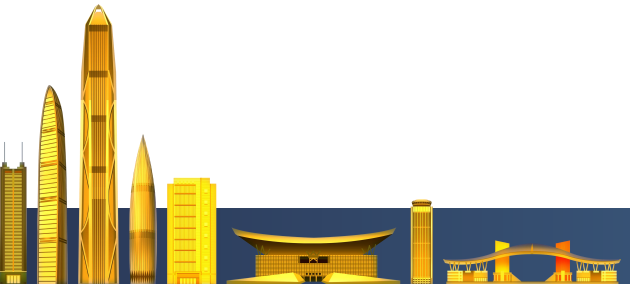
Feature parity with Genesis

Core physics	Genesis	Nexus
Rigid bodies & contacts	●	●
Large scenes (thousands of bodies)	◐	●
Articulations, limits & motors	●	●
MPM	●	●
Two-way rigid ↔ MPM	●	◐
FEM / soft bodies	●	◐
SPH, PBD (cloth, liquids)	●	○
Differentiable	◐	○

Public API	Genesis	Nexus
Python API	●	●
Batched environments	●	●
PyTorch tensor interop	●	○
Native Rust API	○	●
2D simulation	○	●
Runs in the browser	○	●
Determinism (incl. MPM)	◐	●

Non-physics	Genesis	Nexus
URDF / MJCF	●	●
USD	●	○
Mesh formats	●	◐
Interactive viewer	●	●
RGB / depth / seg. cameras	●	●
Ray tracing	●	●
IMU, lidar, tactile	●	○

● Available ◐ Partial / WIP ○ Missing



Roadmap

Work in progress

- P AVBD solver: 110k bodies at ~7ms
- P Two-way MPM ↔ rigid coupling
- P FEM soft bodies
- P Implicit MPM integrator
- P Sleeping

Planned

- A PyTorch interop
- A Gymnasium-style API
- P Continuous collision detection
- N USD import
- N IMU & lidar sensors

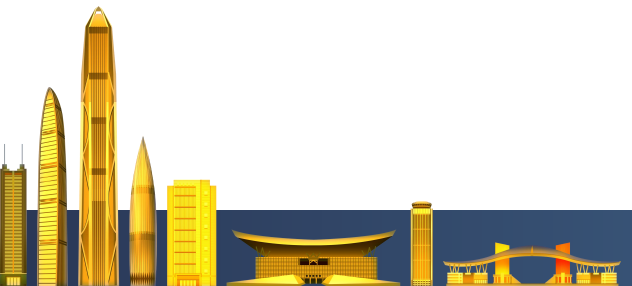
Long term

- P SPH fluids, PBD cloth
- P Differentiable simulation
- A Multi-GPU
- N Batch rendering for vision policies
- N Blender add-on

P Core physics

A Public API

N Non-physics (formats, sensors, tools)



03

Performance & Benchmarks

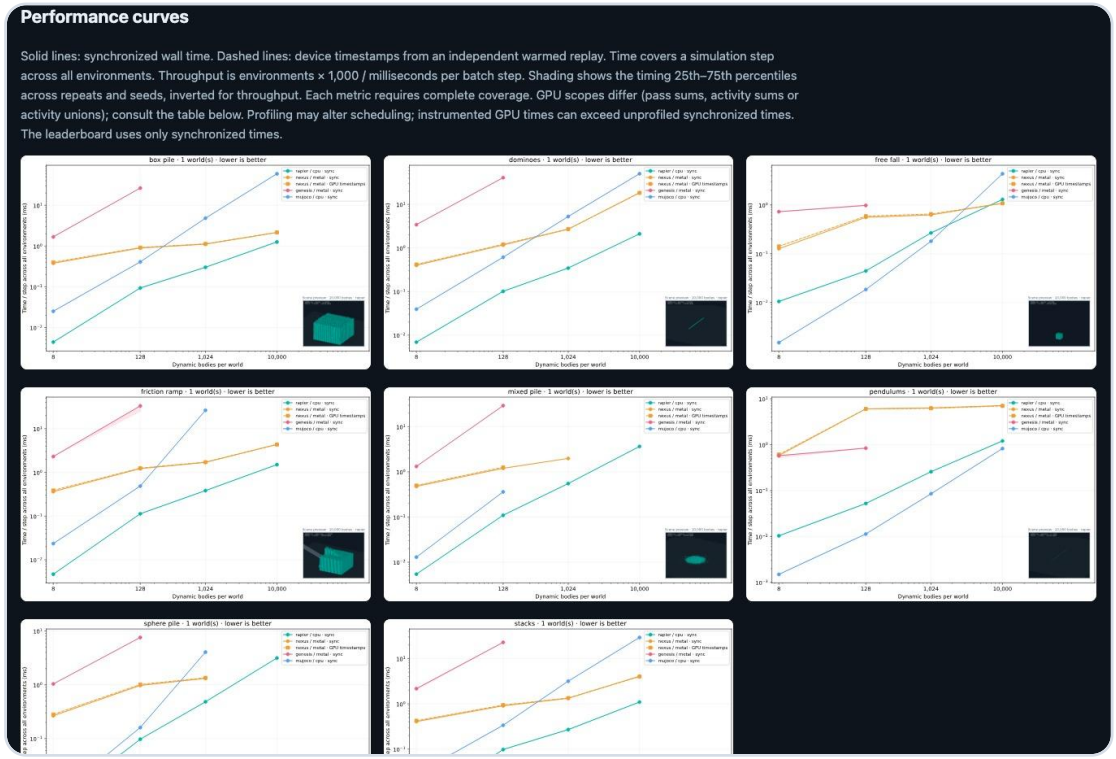


Benchmarking: PhysMark

Multiphysics benchmarking platform: **https://physmark.dimforge.com**

1. **One scene, every engine**
   One JSON scene: explicit mass, inertia, friction, timestep
2. **Isolated engine modules**
   One isolated module per physics engine backend
3. **Automated runs**
   All benchmarks run automatically and sequentially:
   `cargo run --release -- run --suite <suite>.toml`
4. **Illustrated report**
   Curves, leaderboard, screenshots and videos

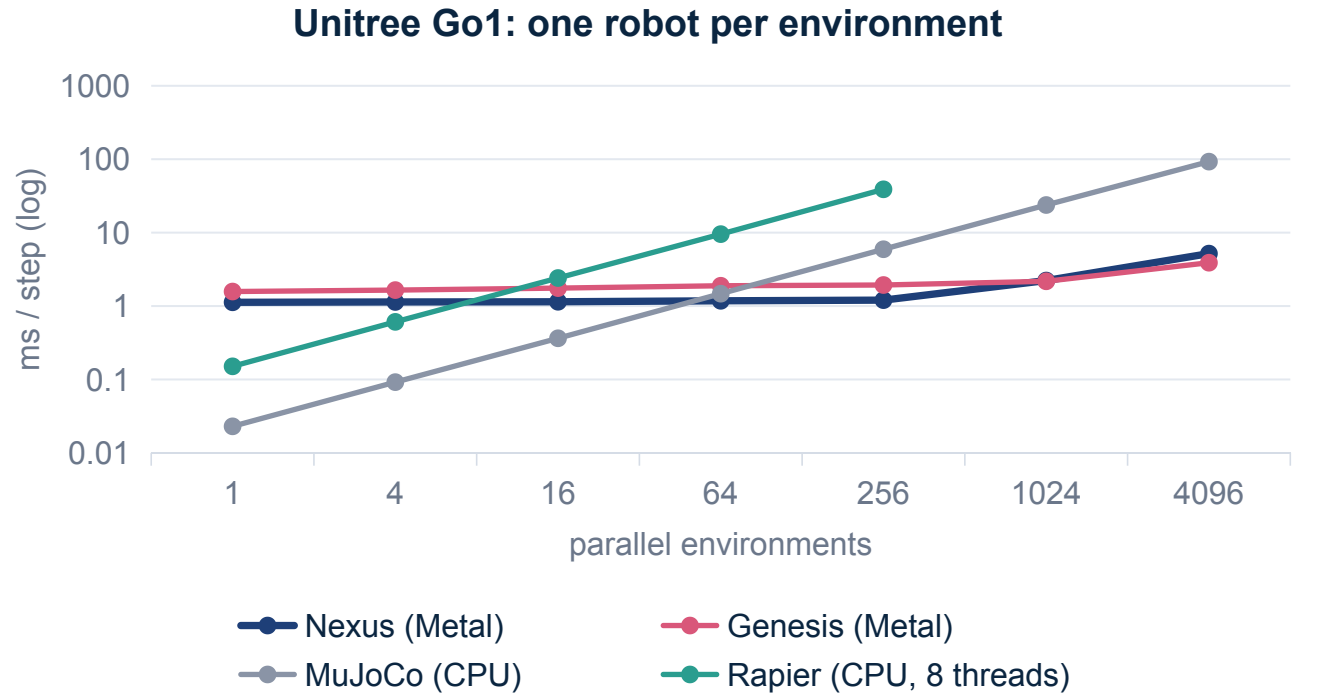
Engines: Nexus, Rapier, Genesis, Newton, MuJoCo, MJX



Generated report: rigid-body scenes from 8 to 10,000 bodies, one world

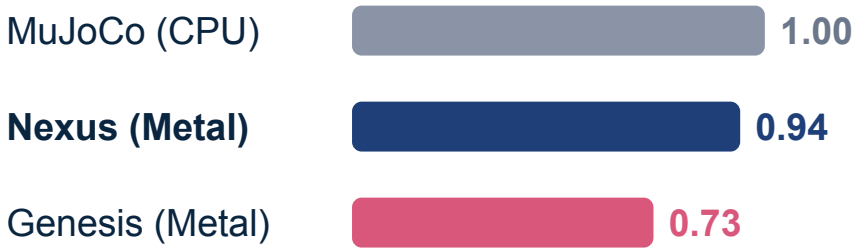
PhysMark: first results

Nexus leads Genesis up to 256 robots and on MPM overall; Genesis pulls ahead at 1024+ robots



M4 Max, median of 3 runs, lower is better. Rapier stops at 256: 120 s CPU budget reached.

Robot score vs MuJoCo (higher is better)

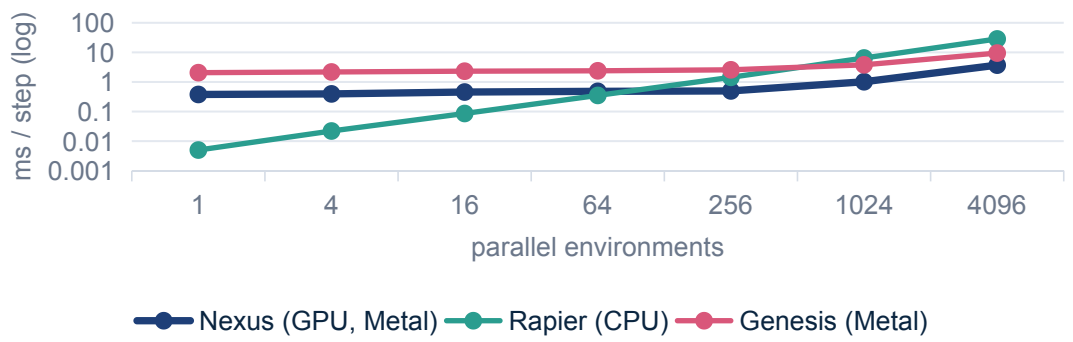


1.33×
Nexus vs Genesis on MPM (elastic, sand, dam break)

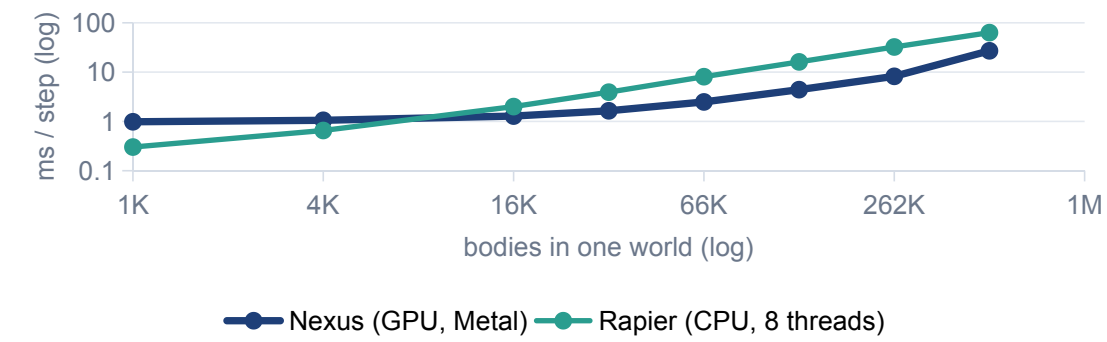
3.7 ms
4096 stacks of 8 boxes (Genesis 9.5, Rapier 28.5)

Newton and MJX need CUDA: not in this Mac round. Sanity-gated, not accuracy-matched.

Nexus or Rapier?



PhysMark, M4 Max, median ms/step. Crossover near 100 environments.



PhysMark, M4 Max, median ms/step. Nexus: codex/avbd-performance branch.

Rapier (CPU) when…

- One world below ~8K bodies (see right chart)
- You need CCD, scene queries, events, character controllers
- No GPU, or cross-platform determinism required

Nexus (GPU) when…

- Hundreds+ of environments (RL, data generation): 4096 stacks in 3.7 ms vs 28 ms
- Huge scenes: 110k-brick ring ~10–19 ms vs 75 ms (Rapier, 16 threads)
- Continua (MPM), or data that must stay on the GPU (input to AI kernels, rendering, etc.)

04

Nexus use-cases



Use-case: reinforcement learning with Zealot

A humanoid whole-body-control training stack written entirely in Rust, on top of Nexus



Real Unitree G1
Video: Xavier Tao - 1ms.ai



Training environment in Nexus



Live demo: run the policy in your browser

~71k
env-steps/s, 4096 G1s, RTX 5090

100%
Rust: env, PPO, deployment

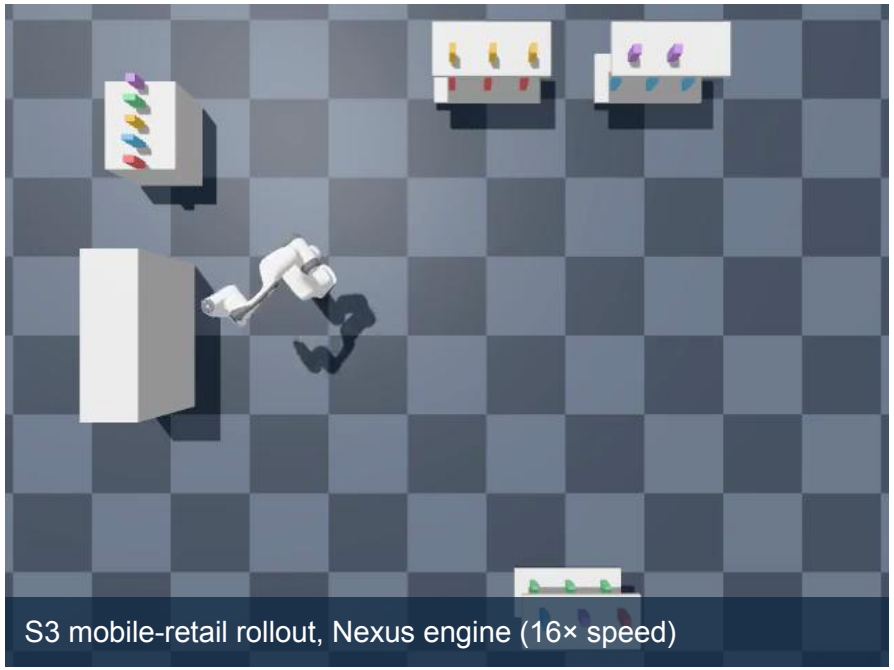
1 source
SPIR-V, PTX and MSL builds

- Unitree G1: walking, push recovery, teleop pick & place, dancing
- PPO trainer in Rust (rsl_rl port) on vortx + khal, like the physics
- Physics substeps, observations and policy run as one GPU workload
- Checked sim-to-sim against MuJoCo, Isaac and Genesis; deployed on a real G1 through LeRobot

github.com/haixuanTao/zealot · by Haixuan Xavier Tao

Use-case: AISLE

Coding agents build robot dataflows; Nexus is one of three interchangeable physics engines



Overhead camera of an AISLE episode: a mobile arm navigating the store shelves, simulated by Nexus on Metal.

Coding agents
Claude Code, Codex

→

Typed dora-rs dataflows
validator, safety guard, scorer

→

Engine-neutral scene
--sim-engine

→

- Genesis
- **Nexus**
- Rapier

What Nexus brings to AISLE

- **pip wheel** for macOS, Linux and Windows (dimforge-nexus3d)
- **Sensor cameras:** RGB, metric depth, per-body segmentation
- **GPU MPM** for lab powder handling

github.com/moxin-org/aisle

GOSIM

GOSIM Shenzhen 2026

Thanks!

Code: github.com/dimforge/nexus
Online demos & user guide: nexus.dimforge.com
Python: pip install dimforge-nexus3d
Benchmarks: physmark.dimforge.com



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

开源机器人

Open Source Robotics

